DATABASE NORMALIZATION

Student Practice Worksheet | UNF → 1NF → 2NF → 3NF → BCNF → 4NF → 5NF

Name: Fahmida mahar

Roll No: 0043_sap_id 5000000393

Date: 13/04/2026

Instructions

Read each section carefully. Each normal form builds on the previous. Show all your work in the answer boxes provided.

Functional Dependency Notation: X → Y means 'X determines Y' (knowing X tells you Y).

GIVEN: Unnormalized Relation (UNF)

The following relation stores data about students, courses they are enrolled in, instructors, and departments. Study it carefully — it contains all the problems normalization is designed to solve.

Relation Name: ENROLLMENT

Primary Key (Composite): *(StudentID, CourseID)*

StudentID	StudentName	CourseID	CourseName	InstructorID	InstructorName	InstructorPhone	DeptID	DeptName	Grade
S101	Ali Khan	CS101	Database Systems	I01	Dr. Sana	0300-1234567	D01	Computer Science	A
S101	*Ali Khan*	CS202	Data Structures	I02	Dr. Bilal	0301-9876543	*D01*	*Computer Science*	B+
S102	Sara Ahmed	*CS101*	*Database Systems*	*I01*	*Dr. Sana*	*0300-1234567*	*D01*	*Computer Science*	A-
S102	*Sara Ahmed*	CS303	Software Eng.	I03	Dr. Fatima	0302-5551234	D02	Software Eng.	B
S103	Usman Raza	*CS202*	*Data Structures*	*I02*	*Dr. Bilal*	*0301-9876543*	*D01*	*Computer Science*	C+
S103	*Usman Raza*	*CS303*	*Software Eng.*	*I03*	*Dr. Fatima*	*0302-5551234*	*D02*	*Software Eng.*	A

Problems in this UNF relation:

- Data Redundancy: Student name and instructor details repeat for every enrolled course.
- Update Anomaly: Changing an instructor's phone requires updating multiple rows.
- Insert Anomaly: Cannot add a new course unless a student is already enrolled in it.
- Delete Anomaly: Deleting the last student from a course removes all course information.

Step 1 — First Normal Form (1NF)

1NF Definition:

A relation is in First Normal Form (1NF) if: (1) All attributes contain only atomic (indivisible) values — no repeating groups or multi-valued attributes. (2) Each column contains values of a single type. (3) Each row is unique (a primary key exists).

Rules to check for 1NF:

1. Eliminate any repeating groups (multiple values in one cell).
2. Ensure every cell has exactly ONE value.
3. Identify and declare a primary key (single or composite).

✅ The ENROLLMENT table above already satisfies 1NF (all cells are atomic, no multi-valued attributes). Your task is to confirm this and list the functional dependencies.

Q1. Is the ENROLLMENT relation in 1NF? Justify by checking each 1NF rule:

- No repeating groups each cell has only one value.
- Atomic values all data is indivisible (no multiple values in one field).
- Same data type in each column each column stores one type of data. .

Q2. List ALL Functional Dependencies (FDs) you can identify from the UNF relation. Use notation: X → Y

- StudentID StudentName
- CourseID CourseName, InstructorID
- InstructorID InstructorName, InstructorPhone, DeptID
- DeptID DepName
- StudentID, CourseID) Grade

Q3. What is the Primary Key of this 1NF relation? Why is a composite key needed here?

Primary Key: (StudentID, CourseID)

One student can take many courses.   One course has many students

StudentID alone is not unique   CourseID alone is not unique

Step 2 — Second Normal Form (2NF)

Partial Dependency — Definition:

A Partial Dependency occurs when a non-key attribute is functionally dependent on PART of a composite primary key, rather than the WHOLE key. Example: If the primary key is (A, B) and attribute C depends only on A (not on both A and B), then C is partially dependent on the key.

2NF Definition:

A relation is in Second Normal Form (2NF) if: (1) It is already in 1NF, AND (2) Every non-key attribute is fully functionally dependent on the ENTIRE primary key — i.e., no partial dependencies exist. (Only applies when the primary key is composite.)

Primary Key: (StudentID, CourseID) — Identify partial dependencies:

StudentID StudentName (StudentName depends only on StudentID, NOT on CourseID)

CourseID CourseName, InstructorID, InstructorName, InstructorPhone, DeptID, DeptName

(StudentID, CourseID) Grade Full dependency Grade depends on BOTH parts

Q4. Circle/Highlight the partial dependencies above. Explain in your own words WHY they violate 2NF:

- **Partial Dependencies**: StudentID StudentName CourseID CourseName

, InstructorID, InstructorNameInstructorPhone, DeptID, DeptName

Why violate 2NF?

Because these attributes depend on only part of the composite key, not the whole key.

To convert to 2NF: Decompose the relation into separate tables, each with its own single-key or appropriate key.

Q5. Write the decomposed 2NF tables below. Give each table a name and list its attributes. Underline th e primary key in each table:

STUDENT *StudentID*, StudentName

COURSE *CourseID*, CourseName, InstructorID, InstructorName, InstructorPhone, DeptID.

ENROLLMENT *StudentID, CourseID*, Grade

Expected 2NF Tables (fill in the blank templates below):

Table 1: STUDENT (StudentID, StudentName)

Table 2: COURSE (*CourseID*, CourseName, InstructorID, InstructorName, InstructorPhone, DeptID, DeptName)

Table 3: ENROLLMENT (*StudentID*, *CourseID*, Grade)

Step 3 — Third Normal Form (3NF)

Transitive Dependency — Definition:

A Transitive Dependency occurs when a non-key attribute (C) depends on another non-key attribute (B), which itself depends on the primary key (A). In other words: A → B and B → C, so A → C transitively. C is NOT directly dependent on the key — it is 'hidden behind' B.

3NF Definition:

A relation is in Third Normal Form (3NF) if: (1) It is already in 2NF, AND (2) No non-key attribute is transitively dependent on the primary key — i.e., every non-key attribute depends DIRECTLY on the primary key and nothing else.

Examine the COURSE table from your 2NF result:

COURSE (CourseID, CourseName, InstructorID, InstructorName, InstructorPhone, DeptID, DeptName)

Transitive Dependencies hiding here:

CourseID InstructorID (direct)   BUT: InstructorID InstructorName, InstructorPhone

DeptID DeptName (DeptName depends on DeptID, not directly on CourseID)

Q6. Identify and explain the transitive dependencies in the COURSE table. Why do they violate 3NF?

Transitive Dependencies:

- CourseID InstructorID InstructorName, Instructor Phone CourseID DeptID DeptName

Why they violate 3NF?

- Because non-key attributes (InstructorName, InstructorPhone, DeptName) depend on

other non-key attributes instead of directly on CourseID (primary key).

Decompose further to eliminate transitive dependencies.

Q7. Write the final 3NF tables below. Show all tables (including those already correct from 2NF):

- **STUDENT** (*StudentID*, StudentName)
- **COURSE** (*CourseID*, CourseName, InstructorID, DeptID)
- **INSTRUCTOR** *InstructorID*, InstructorName, InstructorPhone)
- **DEPARTMENT** (*DeptID*, DeptName)
- **ENROLLMENT** *StudentID*, *CourseID*, Grade)
- All transitive dependencies are separated into their own tables.

Expected 3NF Tables (complete the blanks):

Table 1: STUDENT (StudentID, StudentName)

Table 2: COURSE (CourseID, CourseName, InstructorID, DeptID)

Table 3: INSTRUCTOR (InstructorID, InstructorName, InstructorPhone)

Table 4: DEPARTMENT **(** DeptID , DeptName)

Table 5: ENROLLMENT (StudentID, CourseID, Grade)

Step 4 — Boyce–Codd Normal Form (BCNF)

BCNF Definition:

A relation is in Boyce–Codd Normal Form (BCNF) if: For every non-trivial functional dependency X → Y, X must be a superkey (a key that uniquely identifies every row). BCNF is a stricter version of 3NF — a relation can be in 3NF but NOT in BCNF when there are overlapping candidate keys.

3NF vs BCNF Difference	Classic BCNF Violation Example
3NF allows a non-key attribute to determine part of a key IF it is part of some candidate key. BCNF does NOT allow this — every determinant must be a superkey, no exceptions.	*TEACHING (Student, Subject, Teacher)* Each teacher teaches only one subject: Teacher → Subject Candidate Keys: {Student,Subject}, {Student,Teacher} → Teacher is a determinant but NOT a superkey. BCNF violated!

Q8. Check each of your 3NF tables: Are they in BCNF? For each table, state its candidate keys and verify that every determinant is a superkey.

- **STUDENT (StudentID, StudentName):**

Candidate Key: StudentID FD: StudentID StudentName StudentID is superkey **BCNF satisfied**

- **COURSE (CourseID, CourseName, InstructorID, DeptID):**

Candidate Key: CourseID FD: CourseID all attributes tudentID is superkey **BCNF satisfied**

- **DEPARTMENT (DeptID, DeptName)**

Candidate Key: DeptID FD: DeptID DeptName Superkey determinant **BCNF satisfied**

- **INSTRUCTOR (InstructorID, InstructorName, InstructorPhone)**

Candidate Key: InstructorID FD: InstructorID all attribute Superkey determinant **BCNF satisfied**

- **ENROLLMENT (StudentID, CourseID, Grade)**

Candidate Key: (StudentID, CourseID) FD: (StudentID, CourseID) Grade Superkey determinant **BCNF satisfied**

Q9. If any table violates BCNF, decompose it and write the new BCNF tables here:

- **BCNF Decomposition:**

All 3NF tables already satisfy BCNF. No table violates BCNF because:

Every determinant is a superkey in each table.

No functional dependency exists where X is not a key

- **No decomposition needed.**

All relations are already in **BCNF**, so the tables remain the same.

Step 5 — Fourth Normal Form (4NF)

Multi-Valued Dependency (MVD) — Definition:

A Multi-Valued Dependency (MVD) X ↠Y exists in a relation when, for each value of X, there is a set of values of Y that is independent of all other attributes in the relation. In other words, X can have multiple values of Y and multiple values of Z independently — creating a 'Cartesian product' effect between Y and Z.

4NF Definition:

A relation is in Fourth Normal Form (4NF) if: (1) It is in BCNF, AND (2) It contains no non-trivial multi-valued dependencies. If X ↠Y holds in a relation with attributes (X, Y, Z), then either X is a superkey OR the MVD is trivial (Y is a subset of X, or X ∪ Y covers all attributes).

4NF Example Scenario:

Suppose an instructor can teach multiple courses AND speak multiple languages, independently:

INSTRUCTOR_SKILLS (InstructorID, CourseTaught, Language)

Dr. Sana ↠ {CS101, CS202} (independent of language)

Dr. Sana ↠ {Urdu, English} (independent of course)

This causes spurious tuples. To fix: split into INSTRUCTOR_COURSES(InstructorID, CourseTaught) and INSTRUCTOR_LANGUAGES(InstructorID, Language).

Q10. Explain in your own words what a Multi-Valued Dependency is and how it differs from a regular Functional Dependency.

- **Multi-Valued Dependency (MVD):**

A Multi-Valued Dependency happens when one attribute (X) is linked with multiple independ

values of another attribute Y**.** These values are not related to each other.

F**unctional Dependency (FD):** A Functional Dependency means one attribute X determiv nos

only one single value of another attribute Y.

Q11. Suppose the COURSE relation had: CourseID ↠Textbook and CourseID ↠LabRoom (each course may use multiple textbooks AND multiple lab rooms independently). Write the 4NF decomposition of this scenario:

4NF Decomposition: **MVDs:** CourseID Textbook, CourseID LabRoom

This means textbooks and lab rooms are independent **violates 4NF**

Decomposition into 4NF Tables: **COURSE_TEXTBOOK** (*CourseID*, Textbook)

COURSE_LABROOM (*CourseID*, LabRoom)

COURSE (optional main table) (*CourseID*, CourseName, InstructorID, DeptID)

We split the relation to remove independent multi-valued attributes.

Step 6 — Fifth Normal Form (5NF)

Join Dependency — Definition:

A Join Dependency (JD) ★{R1, R2, ..., Rn} holds on a relation R if R can be perfectly reconstructed (losslessly) by joining R1, R2, ..., Rn. A join dependency is non-trivial if no single Ri equals the full relation R. Decomposing based on FDs or MVDs may still leave subtle join dependencies that only 5NF can resolve.

5NF / PJNF Definition:

A relation is in Fifth Normal Form (5NF), also called Project-Join Normal Form (PJNF), if: (1) It is in 4NF, AND (2) Every non-trivial join dependency in the relation is implied by the candidate keys. In practical terms, no further lossless decomposition is possible without losing information.

5NF Classic Example — Supplier–Part–Project:

SPP (Supplier, Part, Project): A supplier supplies a part to a project — only if:

- The supplier supplies that part (Supplier ↔ Part)
- The supplier works on that project (Supplier ↔ Project)
- The project uses that part (Project ↔ Part)

5NF decomposition → SP(Supplier, Part) + SProject(Supplier, Project) + PP(Project, Part)

Q12. Explain the difference between 4NF and 5NF. When do you need to go to 5NF even after achieving 4NF?

4NF: Removes multi-valued dependencies (MVDs) ensures no attribute has independent multiple values
5NF: Removes join dependencies ensures tables cannot be further split without losing information
When do we need 5NF? Even after achieving 4NF, we go to 5NF when data can still be
broken into smaller tables and correctly reconstructed using joins without losing information
especially in complex many-to-many relationships

Q13. Given the relation: COURSE_AUTHOR_TEXTBOOK (CourseID, AuthorID, TextbookID) — where a course can have multiple authors and multiple textbooks, but an author's textbook is only used in a course if all three conditions hold — decompose this into 5NF tables:

5NF Decomposition Relation: COURSE_AUTHOR_TEXTBOOK (CourseID, AuthorID, TextbookID)
This is a join dependency problem, so we split into smaller relations.
5NF Decomposition COURSE_AUTHOR: (*CourseID*, AuthorID)
COURSE_TEXTBOOK: (*CourseID*, TextbookID).
AUTHOR_TEXTBOOK (*AuthorID*, TextbookID).
The original relation is decomposed into 3 tables to remove redundancy and ensure lossless join

Summary — Fill in the Normalization Reference Table

Complete the summary table below based on everything you have learned:

Normal Form	Requirement	What It Eliminates	Key Concept	Your Example Table (from this worksheet)
1NF	Atomicvalues, no repeating	Repeating groups	Atomic data	ENROLLMENT
2NF	1NF nopartial dependency	Partial dependency	Full key dependency	STUDENT, COURSE, ENROLLMENT
3NF	2NF + no transitive	Transitive dependency	Direct dependency only	STUDENT, COURSE, INSTRUCTOR, DEPARTMENT
BCNF	Every determinant	Functional anomalies	Functional anomalies	All 3NF tables
4NF	BCNF + no MVD	Multi-valued dependency	Independent multi-values	COURSE_TEXTBOOK, COURSE_LABROOM
5NF	4NF + no join dependency	Join redundancy	Lossless decomposition	COURSE_AUTHOR_TEXTBOOK split tables

Q14. In your own words, why is normalization important in database design? What are the trade-offs?

Normalization is important because it organizes data properly, reduces duplication, and prevents data problems like update, insert, and delete anomalies. It also keeps the database consistent and reliable.

However, the trade-off is that normalized databases often need more tables and joins, which can make queries slower and more complex to write and manage.

Marks: __________ / 5

Instructor Signature: ______________________